CODING PRACTICES IN ASP.NET CORE MVC

Topics Covered

1. Separation of Concerns (Models, Views, Controllers)
2. Consistent Naming Conventions
3. DRY (Don't Repeat Yourself) and Reusable Components
4. Folder Structure and Organization

Learning Objectives

At the end of this lesson, students should be able to:

- Explain why coding practices matter in software development.
- Describe the responsibilities of Models, Views, and Controllers in ASP.NET Core MVC.
- Apply common C# naming conventions to classes, methods, properties, variables, and parameters.
- Identify unnecessary code duplication and improve code reuse using methods, services, layouts, and partial views.
- Organize an ASP.NET Core MVC project using a clear and maintainable folder structure.
- Evaluate a short code sample and recommend improvements based on the four practices.

Introduction: What Are Coding Practices?

Coding practices are guidelines and techniques developers follow when writing, organizing, and maintaining software. Good practices make code easier to read, test, debug, extend, and maintain. A program may work even when its code is poorly organized, but the cost of changing it usually increases as the application grows.

Imagine developing SpendSmart, an expense-tracking web application. Users can add expenses, edit records, view their spending history, and monitor a budget. If all code is placed in one large file, it becomes difficult to find features, change the interface safely, avoid repeated logic, and collaborate with others. The four practices in this module help prevent these problems.

Topic 1: Separation of Concerns (Models, Views, Controllers)

1.1 What is separation of concerns?

Separation of concerns is a software design principle in which different parts of an application have different responsibilities. A useful analogy is a restaurant: a waiter receives an order, the kitchen prepares the food, and the menu presents available dishes. They cooperate, but each has a distinct role.

In ASP.NET Core MVC, the main components are Model, View, and Controller. Keeping their responsibilities clear makes the application easier to understand and maintain.

1.2 The Model

A Model represents application data and may include validation rules or domain-related behavior. For SpendSmart, an Expense model might describe an expense's ID, amount, description, date, and category.

```
namespace SpendSmart.Models
{
    public class Expense
    {
        public int Id { get; set; }
        public decimal Amount { get; set; }
        public string Description { get; set; } = "";
        public DateTime Date { get; set; }
        public string Category { get; set; } = "";
    }
}
```

What the code means:

- Expense is a class describing the structure of an expense record.
- Id identifies a record; Amount stores a monetary value.
- Description and Category store text; Date stores a date and time value.

- The model defines the structure of the data. It does not automatically save data to a database; data access requires a database context or another data-access mechanism.

1.3 The View

A View presents information to the user. In ASP.NET Core MVC, views commonly use Razor syntax in .cshtml files. A view should focus on presentation rather than opening database connections or performing complex business calculations.

```
@model IEnumerable<SpendSmart.Models.Expense>

<h2>My Expenses</h2>

<table class="table">
    <thead>
        <tr>
            <th>Description</th>
            <th>Amount</th>
            <th>Date</th>
            <th>Category</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var expense in Model)
        {
            <tr>
                <td>@expense.Description</td>
                <td>@expense.Amount.ToString("C")</td>
                <td>@expense.Date.ToShortDateString()</td>
                <td>@expense.Category</td>
            </tr>
        }
    </tbody>
</table>
```

The @model directive declares the type of data expected by the view. IEnumerable<Expense> means the view expects a collection of expense objects. The foreach loop visits each record and displays its values. The expression @expense.Description outputs the current expense's description.

1.4 The Controller

A Controller receives requests, coordinates the response, and selects the view or other result to return. For example, an ExpensesController Index action can prepare a collection of expenses and pass it to the view.

```
using Microsoft.AspNetCore.Mvc;
using SpendSmart.Models;

namespace SpendSmart.Controllers
{
    public class ExpensesController : Controller
    {
        public IActionResult Index()
        {
            var expenses = new List<Expense>
            {
                new Expense
                {
                    Id = 1,
                    Amount = 150,
                    Description = "Lunch",
```

```
                    Date = DateTime.Today,
                    Category = "Food"
                },
                new Expense
                {
                    Id = 2,
                    Amount = 80,
                    Description = "Bus Fare",
                    Date = DateTime.Today,
                    Category = "Travel"
                }
            };

            return View(expenses);
        }
    }
}
```

This example uses sample data held in memory; it does not retrieve records from a database. The line return View(expenses) tells MVC to render the Index view and pass the collection to it. By convention, the view is located at Views/Expenses/Index.cshtml.

1.5 How MVC components work together

1. The user requests a page, such as /Expenses/Index.
2. Routing directs the request to the appropriate controller action.
3. The controller obtains or prepares the data, often using a service or database context.
4. The model describes the data being handled.
5. The controller passes the data to a view.
6. The view generates the HTML that the browser displays.

This is a simplified flow. A real application may also involve middleware, model binding, validation, authentication, and data-access services.

1.6 What happens when responsibilities are mixed?

If a controller contains database access, complex calculations, HTML strings, and presentation formatting, it becomes difficult to change or test safely. A change to the page design may require editing code that also performs business operations. Keep each responsibility in the most appropriate component.

Benefits include better maintainability, readability, testability, collaboration, and debugging.

Check your understanding: If the requirement is to change the expense table's colors and layout, which part should normally be changed? Answer: The View and its related CSS.

Topic 2: Consistent Naming Conventions

2.1 What are naming conventions?

Naming conventions are agreed rules for naming classes, methods, properties, variables, parameters, and files. Good names communicate what an item represents or what it does. Meaningful names are especially useful when you revisit code after several weeks or months.

```
decimal x = 500;
decimal monthlyTransportationExpense = 500;
```

Both variables contain the same value, but monthlyTransportationExpense explains the value's purpose. The name x does not provide useful context.

2.2 Common C# naming styles

Style	Common use	Example
PascalCase	Classes, methods, public properties, namespaces	Expense, CalculateTotal(), MonthlyBudget
camelCase	Local variables and method parameters	totalExpense, expenseId
_camelCase	Private fields (common convention)	_monthlyBudget, _context

Example:

```
public class Budget
{
    public decimal MonthlyBudget { get; set; }

    public bool IsOverBudget(decimal totalExpense)
    {
        return totalExpense > MonthlyBudget;
    }
}
```

Budget is a class name, MonthlyBudget is a public property, IsOverBudget is a method, and totalExpense is a parameter. Their names follow common C# conventions and reveal their purpose.

2.3 Naming MVC controllers and views

```
public class ExpensesController : Controller
{
    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Create()
    {
        return View();
    }
}
```

The conventional view locations are Views/Expenses/Index.cshtml and Views/Expenses/Create.cshtml. MVC uses conventions to find views based on the controller and action names. Explicit view paths are possible, but conventional names keep projects predictable.

2.4 Common naming mistakes

Poor: public void calc() | Better: public void CalculateMonthlyExpenses()

Poor: decimal amt = 3000; | Better: decimal monthlyExpense = 3000;

Poor: bool flag = true; | Better: bool isOverBudget = true;

Avoid unexplained abbreviations, vague method names, inconsistent capitalization, and misleading names. C# is case-sensitive, so totalExpense and TotalExpense are different identifiers.

Naming conventions improve readability, maintainability, collaboration, debugging, and consistency. Remember that good naming does not guarantee correct logic, but it makes the logic easier to understand.

Check your understanding: Which is clearer for calculating monthly expenses: Compute() or CalculateMonthlyExpenses()? The second name communicates the purpose.

Topic 3: DRY and Reusable Components

3.1 What does DRY mean?

DRY stands for Don't Repeat Yourself. It encourages developers to avoid unnecessary duplication of the same logic, rules, or interface content. When a rule is copied into many places, every copy must be maintained, and one forgotten update can cause inconsistent behavior.

3.2 Example of duplicated code

```
public IActionResult Create()
{
    ViewBag.Categories = new List<string>
    {
        "Food", "Travel", "Bills"
    };
    return View();
}

public IActionResult Edit(int id)
{
    ViewBag.Categories = new List<string>
    {
        "Food", "Travel", "Bills"
    };
    return View();
}
```

The category list is repeated. If Education is added, the developer must update both copies. If one is missed, the Create and Edit pages may show different choices.

3.3 Improve the code with a reusable method

```
private List<string> GetCategories()
{
    return new List<string>
    {
        "Food", "Travel", "Bills", "Education"
    };
}

public IActionResult Create()
{
    ViewBag.Categories = GetCategories();
    return View();
}

public IActionResult Edit(int id)
{
    ViewBag.Categories = GetCategories();
    return View();
}
```

Now the category list is defined in one place. For a larger application, categories may come from a database through a service rather than from a hard-coded list.

3.4 Reusable business logic

```
private bool IsOverBudget(
    decimal totalExpense,
    decimal monthlyBudget)
{
    return totalExpense > monthlyBudget;
}
```

A shared method can be used wherever the same budget check is needed. In a larger application, business rules may belong in a dedicated service so multiple controllers can use them. DRY should reduce meaningful duplication without introducing unnecessary abstraction.

3.5 Common reusable components in ASP.NET Core MVC

A. Layout pages

A layout provides the common structure shared by pages, such as a header, navigation bar, footer, and common CSS or JavaScript references. The common layout file is often Views/Shared/_Layout.cshtml.

B. Partial views

A partial view is a reusable section of a page, such as an expense summary card or repeated form fields. For example, Views/Shared/_ExpenseSummary.cshtml can be rendered in more than one view.

```
@model decimal

<div class="card">
    <div class="card-body">
        <h5>Total Expenses</h5>
        <p>@Model.ToString("C")</p>
    </div>
</div>
```

A parent view can render the partial with a compatible model value, for example: <partial name="_ExpenseSummary" model="Model.TotalExpenses" />. This assumes the parent view model has a TotalExpenses property.

C. Helper methods and services

Helper methods and services can centralize shared operations such as calculating totals, checking budgets, or retrieving categories. Services are useful when multiple controllers need the same application logic.

Do not reuse code merely because two blocks look similar. If they have different responsibilities or rules, combining them may make the design more complicated. Reuse is valuable when it makes the code clearer and easier to maintain.

Benefits include less maintenance, consistency, fewer copy-related mistakes, faster development, and clearer code.

Check your understanding: If Create and Edit pages use the same expense form fields, one option is to extract the shared markup into a partial view and render it from both pages.

Topic 4: Folder Structure and Organization

4.1 What is project organization?

Folder structure is the way project files are arranged. A clear structure helps developers find code, understand responsibilities, add features, and collaborate. Think of it as a filing cabinet: labeled folders make documents easier to find than placing every document in one drawer.

4.2 Example of a poorly organized project

```
SpendSmart/
├── Expense.cs
├── ExpensesController.cs
├── Index.cshtml
├── Create.cshtml
├── site.css
├── site.js
├── logo.png
├── ApplicationDbContext.cs
└── Program.cs
```

This may work for a tiny experiment, but it becomes difficult to identify which views belong to a controller or where static resources and database-related files should go.

4.3 Recommended ASP.NET Core MVC structure

```
SpendSmart/
├── Controllers/
│   ├── HomeController.cs
│   └── ExpensesController.cs
├── Models/
│   ├── Expense.cs
│   └── Category.cs
├── Views/
│   ├── Home/
│   │   └── Index.cshtml
│   ├── Expenses/
│   │   ├── Index.cshtml
│   │   ├── Create.cshtml
│   │   ├── Edit.cshtml
│   │   └── Details.cshtml
│   └── Shared/
│       ├── _Layout.cshtml
│       └── Error.cshtml
├── Data/
│   └── ApplicationDbContext.cs
├── Services/
│   └── ExpenseService.cs
├── Migrations/
├── wwwroot/
│   ├── css/
│   │   └── site.css
│   ├── js/
│   │   └── site.js
│   └── images/
│       └── logo.png
└── Program.cs
```

The Services and Migrations folders are shown as optional/common additions: Services is useful when reusable application logic warrants it, and Migrations is used when Entity Framework Core migrations are part of the project.

4.4 What belongs in each folder?

- **Controllers/** Controller classes that handle requests and coordinate responses.
- **Models/** Classes that represent application data and may include validation rules.
- **Views/** Razor .cshtml files organized by controller, plus shared layouts and partial views.
- **Data/** Database context and related data configuration, when the project uses this organization.
- **Services/** Reusable application or business logic, when needed.
- **wwwroot/** Static resources such as CSS, JavaScript, and images. Do not put secrets or private files here.
- **Migrations/** Entity Framework Core migration files, if migrations are used.
- **Program.cs** Application startup, service registration, middleware, and routing configuration.

4.5 Adding a new feature

Suppose a ReportsController is added. Its views would normally go into Views/Reports/. A report-specific model may go in Models/, shared business calculations may go in a suitable service, and report styles may be added to wwwroot/css/ or a dedicated stylesheet. Not every feature needs a new model or service; the structure should fit the actual requirements.

4.6 Common organization mistakes

- Putting every view in one folder instead of grouping views by controller.
- Placing CSS, images, models, and controllers together without a clear structure.
- Making controllers excessively large by placing all business logic inside them.
- Creating many layers and folders for a very small project when they do not add value.
- Storing passwords, connection strings, or sensitive documents in wwwroot.

Start with the standard MVC structure and add folders as the application needs them. The objective is clarity, not the largest possible number of folders.

5. Putting the Four Practices Together

Imagine implementing an Add Expense feature. Separation of concerns means the model defines the expense data, the controller handles the request, and the view displays the form. Naming conventions make identifiers clear, such as ExpenseAmount, Create(), and expenseId. DRY encourages reuse of shared form fields or category retrieval. Folder organization places each file in the appropriate location.

Practice	Application
Separation of concerns	Model, View, and Controller each have clear responsibilities.
Naming conventions	Names are meaningful and follow consistent C# conventions.
DRY and reusable components	Shared logic and interface sections are reused appropriately.
Folder structure	Files are grouped in predictable, relevant folders.